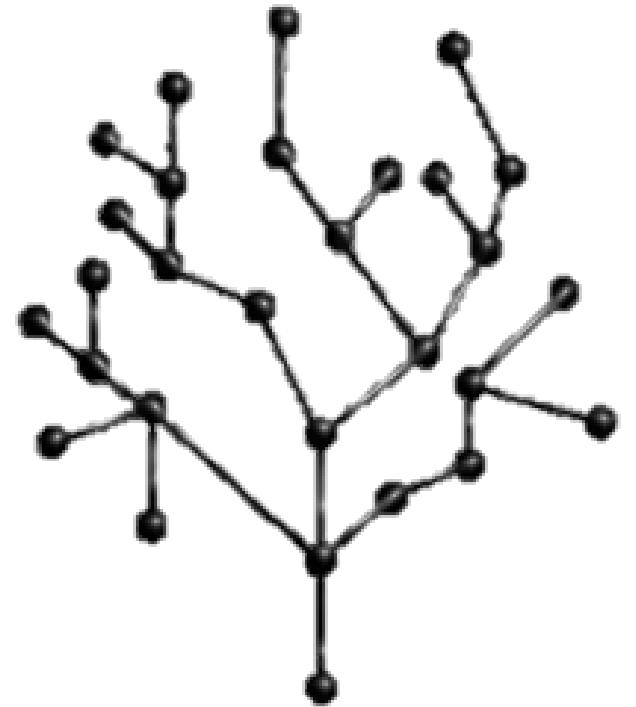




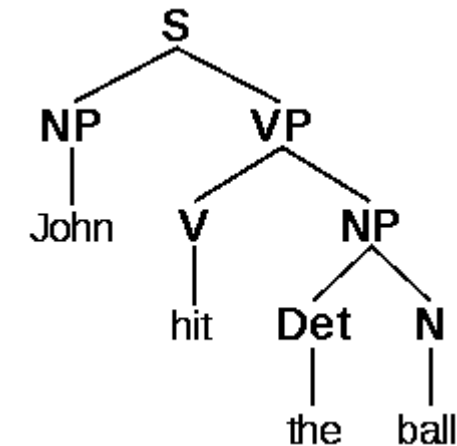
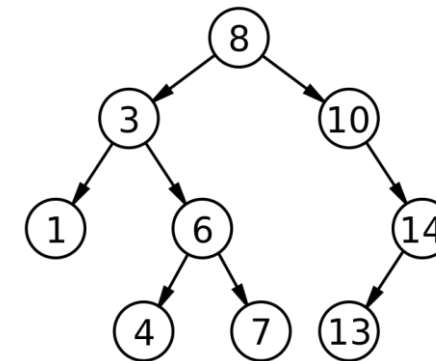
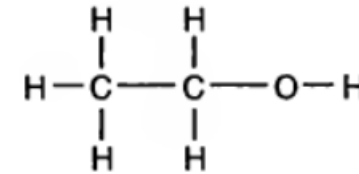
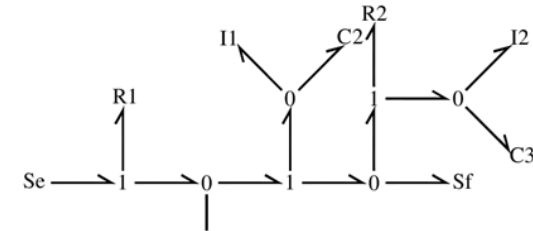
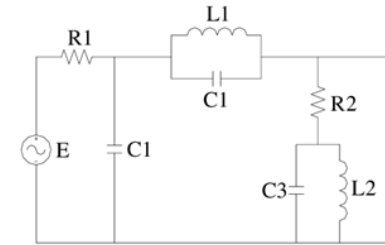
Trees

Mohammed Brahimi



Applications of Trees

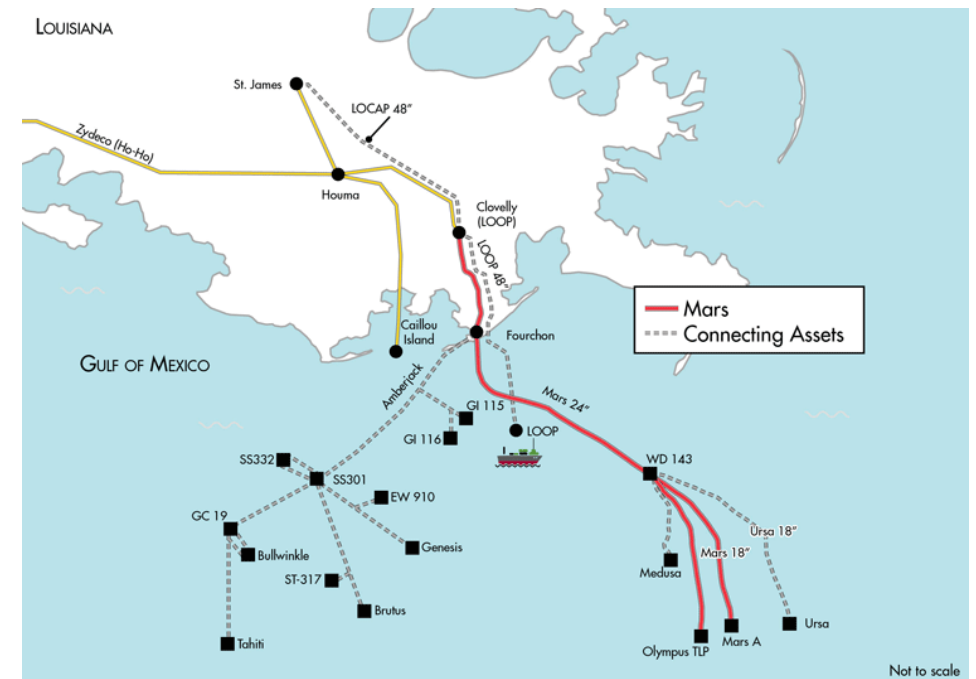
- **Electrical Networks - Gustav Robert Kirchhoff, 1840**
 - Gustav Robert Kirchhoff made significant contributions to the field of electrical networks.
 - He developed models and analyzed interconnected circuits using tree structures.
- **Molecular Enumeration - Arthur Cayley, 1870**
 - Arthur Cayley's work focused on enumerating molecules using tree representations.
 - Trees were used to model and analyze properties of molecules.
- **More Recent Applications**
 - **Computer Science:** Trees are fundamental in algorithms, data structures, and search techniques.
 - **Decision Making:** Tree-based decision models provide optimal choices in various scenarios.
 - **Linguistics:** Syntax trees enable the analysis of language syntax and hierarchical relationships.
 - **Gas Pipeline System Design:** Trees are used to optimize network design and routing.



Applications of Trees - Pipeline Distribution System

- Oil or gas pipeline distribution systems are a key example of artificial trees.
- Economic Consideration:
 - The construction cost of such networks can be exceedingly high.
 - Adopting a tree structure can help minimize unnecessary edges, optimizing the network design.

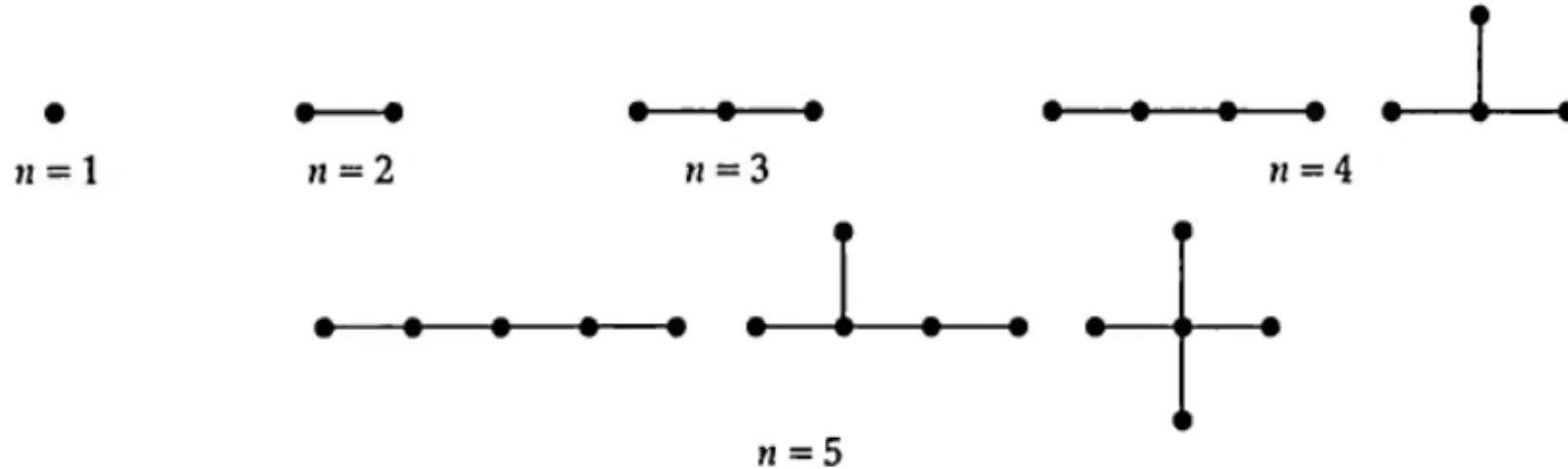
Employing a tree structure can lead to substantial cost reductions, rendering it the most cost-effective design for the network.



Mathematical properties of tree

- Definition

A tree is a connected graph that has no cycles (acyclic).



- The number of vertices is called the **order of tree**.
- The number of edges is called the **size of tree**.

Theorem 1: Equivalent definitions of a Tree

- Let T be a graph with order n (n vertices). Then the following statements are equivalent.
 1. T is a tree.
 2. T contains no cycles, and has $n - 1$ edges.
 3. T is connected, and has $n - 1$ edges.
 4. T is connected, and each edge is a bridge.
 5. Any two vertices of T are connected by exactly one path.
 6. T contains no cycles, but the addition of any new edge creates exactly one cycle.

Proof (1 $\Rightarrow$ 2):

T is a tree $\Rightarrow T$ contains no cycles, and has $n-1$ edges.

Induction on the number of edges

- **Base case:** A tree with one vertex has 0 edges.
- **Induction step:** Removing an edge from T disconnects it into two smaller trees.
 - Removing an edge from a tree (a cycle-free graph) yields two smaller graphs.
 - Each of the smaller graph is also a tree, with one fewer edge than the original tree (removing one edge won't create a cycle).
 - Therefore, the original tree has $n-1$ edges, where n is the number of vertices in the original tree.
- Adding the removed edge back, we conclude the original tree has $n-1$ edges
 - $|E(T_1)| = |V(T_1)| - 1$ (induction hypothesis)
 - $|E(T_2)| = |V(T_2)| - 1$ (induction hypothesis)
 - $|E(T)| = |V(T_1)| - 1 + |V(T_2)| - 1 + 1$ (removed edge is back)
 - $|E(T)| = |V(T)| - 1$ ($|V(T_1)| + |V(T_2)| = |V(T)|$)

Proof (2 $\Rightarrow$ 3):

T contains no cycles and has $n-1$ edges $\Rightarrow T$ is **connected**, and has $n-1$ edges.

- If T is disconnected, then each component of T_i a connected graph with no cycles and hence **(using 2)**.
- The number of vertices in each component exceeds the number of edges by 1.
 - $|E(T_i)| = |V(T_i)| - 1$ **(using 1 $\Rightarrow$ 2)**
- It follows that the total number of vertices of T exceeds the total number of edges by at least 2, contradicting the fact that T has $n - 1$ edges.
 - $|E(T)| = \sum |E(T_i)|$
 - $|E(T)| = \sum |V(T_i)| - k$
 - $|E(T)| = V(T) - k$
 - $|E(T)| < V(T) - 1$
 - **Contradiction with (2)**

(k is the components)

($\sum |V(T_i)| = V(T)$)

(The graph is disconnected $\Rightarrow k > 1$)

Proof (3 $\Rightarrow$ 4):

T is connected, and has $n-1$ edges $\Rightarrow T$ is connected, and **each edge is a bridge.**

- The removal of any edge results in a graph with n vertices and $n - 2$ edges, which must be disconnected by **THEOREM 2 in Week8 Connectivity in graphs ($k = 1$)**.

THEOREM 2

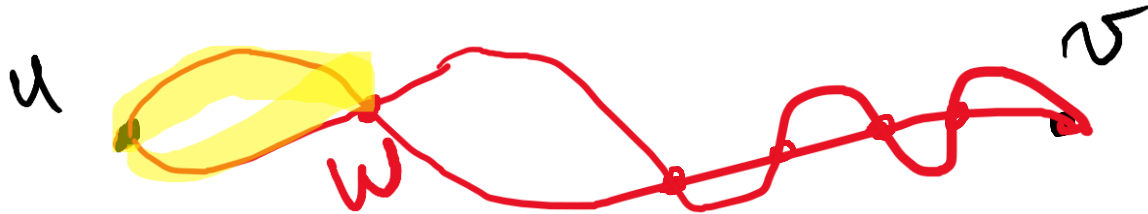
Let G be a simple graph on n vertices. If G has k components, then the number m of edges of G satisfies:

$$n - k \leq m \leq \frac{(n - k)(n - k + 1)}{2}$$

Proof (4 $\Rightarrow$ 5):

T is connected, and each edge is a bridge $\Rightarrow$ Any two vertices of T are connected by **exactly one path**.

- Since T connected, each pair of vertices is connected by at least one path.
- Assume that a given pair of vertices is connected by two paths

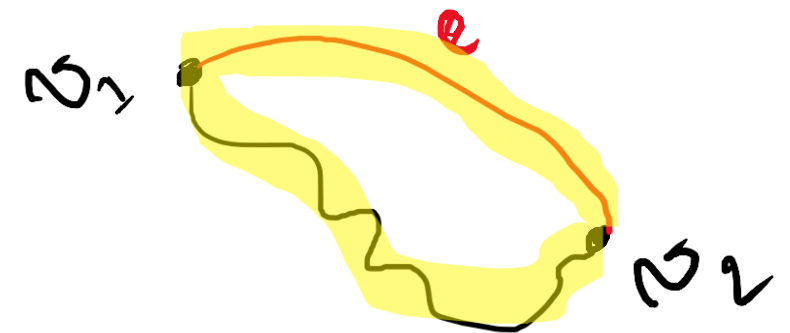
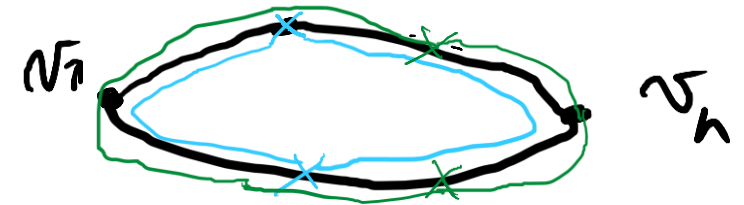


- Then these paths contain a cycle, contradicting the fact that each edge is a bridge
(Cycle cannot contain a bridge).

Proof (5 $\Rightarrow$ 6):

Any two vertices of T are connected by exactly one path $\Rightarrow T$ contains no cycles, but the **addition of any new edge creates exactly one cycle**.

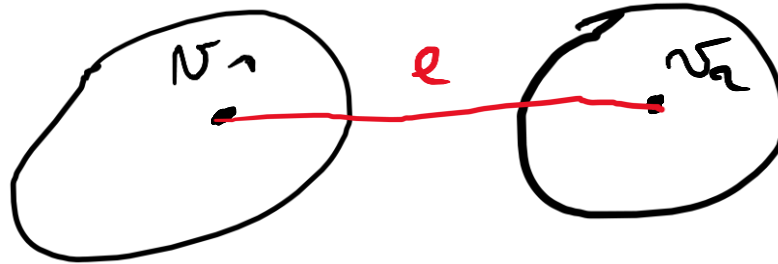
- If T contained a cycle
 - Any two vertices in the cycle are connected by at least two paths, contradicting (5).
- If an edge e is added
 - Since the vertices incident with e are already connected **by exactly one path** in T
 - Add e close the cycle and create exactly one cycle.



Proof (6 $\Rightarrow$ 1):

T contains no cycles, but the addition of any new edge creates exactly one cycle $\rightarrow$
 T is a tree.

- T is a tree $\Leftrightarrow$ A tree is a **connected graph** that has no cycles
- Assume that the graph is disconnected
 - We can add an edge e to connect two components without creating a cycle.



- This is a contradiction with (6) that states that adding any new edge creates exactly one cycle.

Theorem 2:

if $n \geq 2$, any tree on n vertices has at least two end-vertices (with degree =1).

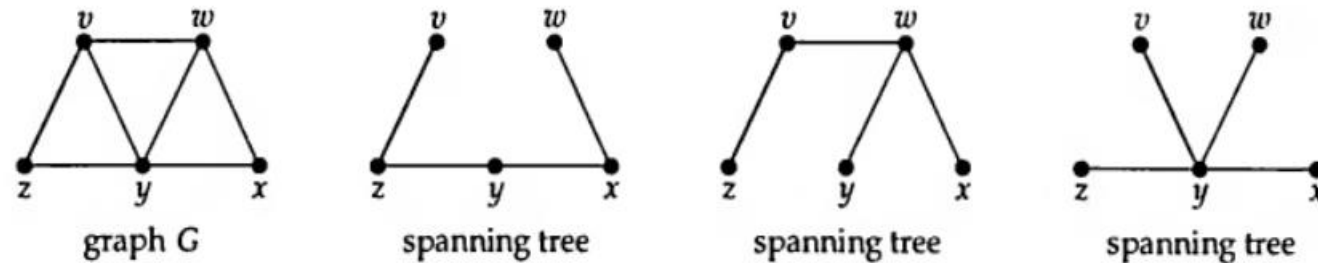
Proof :

- Using the handshaking lemma: $\sum_{i=1}^n d_i = 2|E| = 2(n-1) = 2n-2 \dots \textbf{(1)}$
- Assuming that all vertices have degrees greater 1 ($d_i \geq 2$)
 - $\sum_{i=1}^n d_i \geq 2n$
 - **Contradiction with (1)**
 - **It exists at least one vertex with $d_i = 1$**
- Assume that the graph contains only one vertex with $d_i = 1$
 - $\sum_{i=1}^n d_i \geq 1 + \sum_{i=2}^n 2 = 1 + 2n - 2 = 2n - 1$
 - **Contradiction with (1)**
 - **At least two vertices with degree equal to 1 (end-vertices)**

Spanning Trees

- Definition

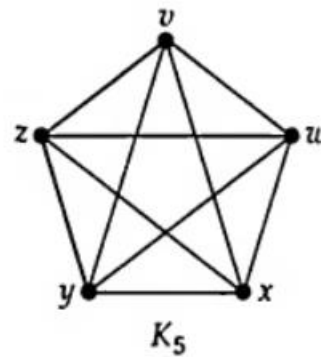
Let G be a connected graph. Then a spanning tree in G is a subgraph of G that includes every vertex and is also a tree.



- The number of spanning trees in a graph can be very large
 - The Petersen graph has 2000 labelled spanning trees.

Constructing a spanning tree

- Choose any cycle and remove any one of its edges;
- Repeat this procedure until no cycles remain.
- Example:
 - Use each method to construct a spanning tree in the complete graph K_5 .



THEOREM 3

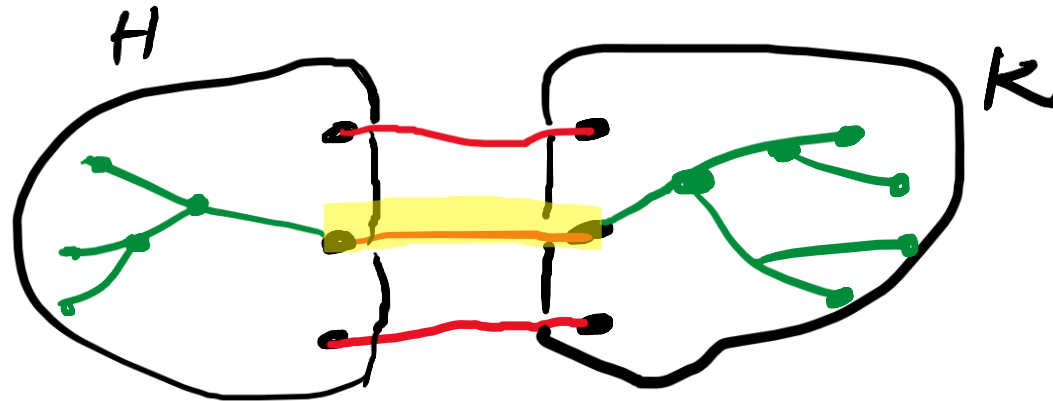
If T is any spanning tree Of a connected graph G , then

- 1. Each cutset of G has an edge in common With T ;*
- 2. Each cycle of G has an edge in common With the complement of T .*

Proof of (1):

Each cutset of G has an edge in common With T

- Let C^* be a cutset of G , the removal of which splits a component of G into two subgraphs H and K .
- Since T is a spanning tree, T must contain an edge joining a vertex of H to a vertex of K and this edge is the required edge.



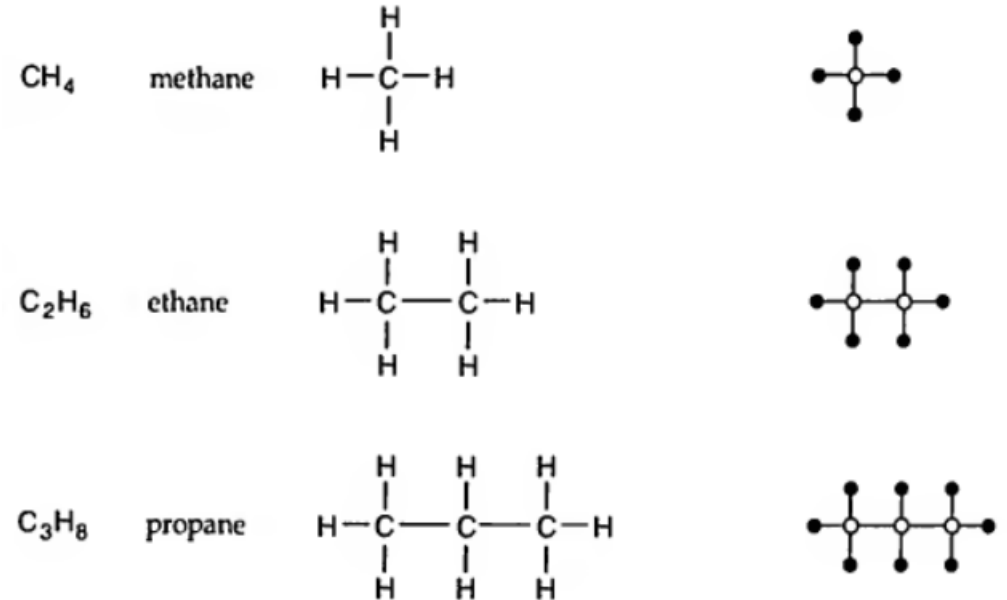
Proof of (2):

Each cycle of G has an edge in common With the complement of T .

- Let C a cycle of G having no edge in common with the complement of T .
- Then C must be contained in T , which is a contradiction.

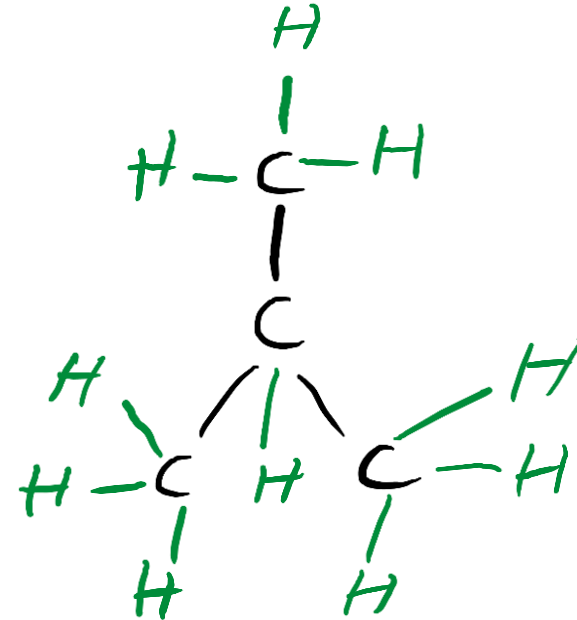
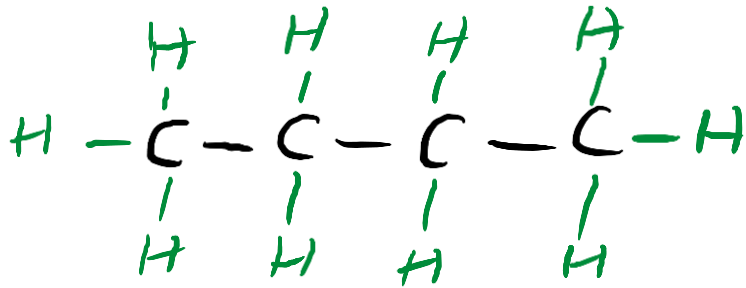
Counting trees

- Graph Enumeration
 - Determining the number of non-isomorphic graphs that **satisfy certain properties**.
- Enumerating alkanes
 - Initiated by Arthur Cayley in the 1850s
 - Enumerating alkanes (C_nH_{2n+2}).
 - Translates into counting trees where each vertex's degree is either 4 or 1.



Counting chemical molecules

- The chemical formula **C_4H_{10}** ($n = 4$).



How many different molecules are there with $\mathbf{C}_n\mathbf{H}_{2n+2}$?

- The graph is connected
- **# of vertices** = $n + (2n + 2) = (3n + 2)$ vertices
- **# of edges** = $\frac{1}{2}\{4n + (2n + 2)\} = (3n + 1)$ edges = **# of vertices** – 1
- T is connected, and has **# vertices** – 1 edges

Then the graph is Tree ([Theorem 1 \(3\)](#))

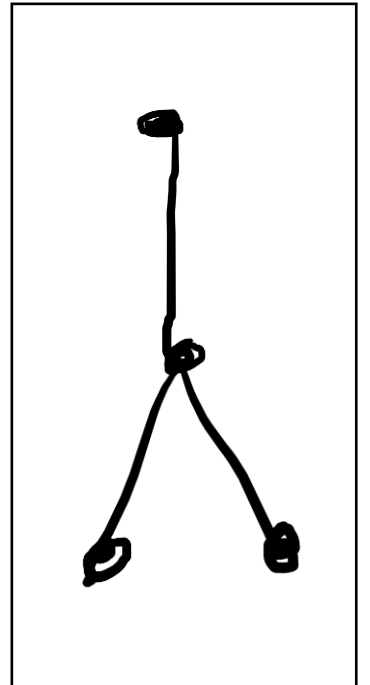
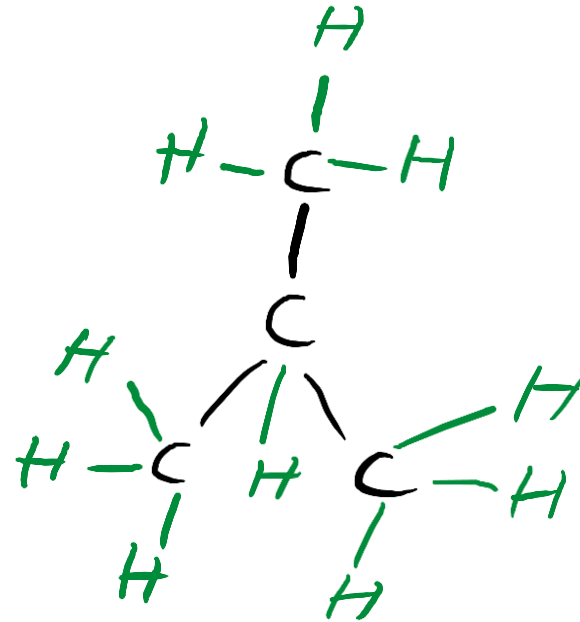
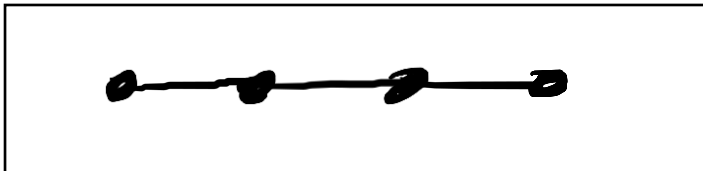
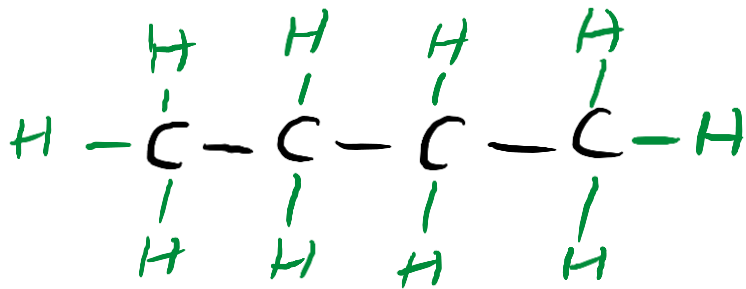
How many different molecules are there with C_nH_{2n+2} ?

- The alkane is determined completely once the carbon atoms are arranged,
 - Hydrogen atoms can be added to ensure each carbon vertex has a degree of 4.
- The problem reduces to that of finding the number of trees with n vertices, each of **degree 4 or less**.

This problem was solved by Cayley in 1875.

Counting chemical molecules

- The chemical formula $\mathbf{C_4H_{10}}$ ($n = 4$).



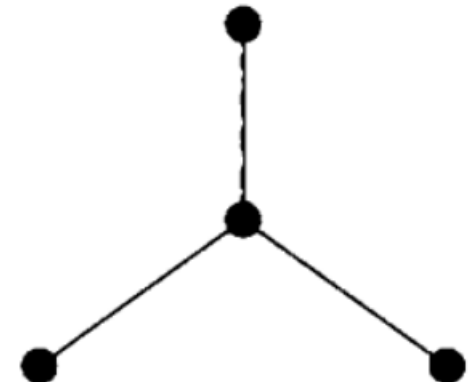
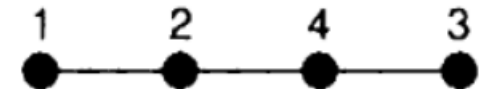
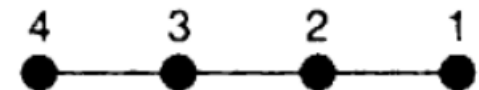
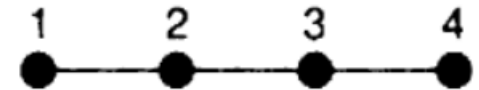
Counting Labelled trees

- Two possible types of trees with 4 vertices:
 - "Path" and "Star".
- **Path Trees**
 - Possible permutations: $4!$ (4 factorial).
 - Accounting for reversals, which don't yield new paths: $\frac{4!}{2} = 12$.
- **Star Trees**
 - Determined by the label of the central vertex.
 - Central vertex can be labelled in **4 different ways**.

Total number of labelled trees with 4 vertices:

$$12 \text{ (Path Trees)} + 4 \text{ (Star Trees)} = 16.$$

What about the general case with n vertices ?



THEOREM 4 (Cayley, 1889)

There are n^{n-2} distinct labelled trees with n vertices.

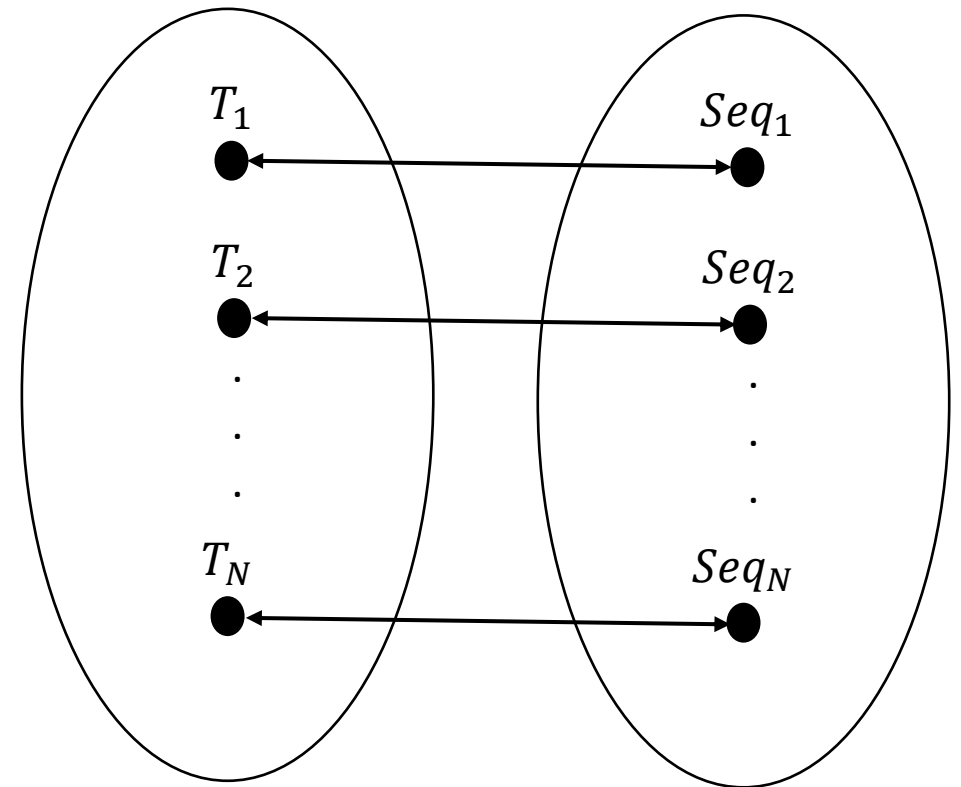
n	1	2	3	4	5	6	7	8	9
Labelled trees	1	1	3	16	125	1296	16807	262144	4782969
Unlabelled trees	1	1	1	2	3	6	11	23	47

Counting unlabelled trees, where vertices aren't distinct, is more complex than counting labelled ones, lacking a simple formula like Cayley's.

First proof:

Proving the Count of Labelled Trees via Prüfer Sequences

- A one-to-one correspondence is established between labelled trees with ' n ' vertices and Prüfer sequences of ' $n - 2$ ' numbers.
- Prüfer sequences $(a_1, a_2, a_3, \dots, a_{n-2})$,
 - a_i is one of the integers $1, 2, 3, \dots, n$ (allowing repetition)
- This method of correlation is credited to H. Prüfer.
- The construction of this correspondence between includes in two parts:
 - Transforming a labelled tree into a Prüfer sequence.
 - Converting a Prüfer sequence back into a labeled tree.



Construction of a Prufer Sequence from a Labelled Tree

- **Step 1**
 - Identify vertices of degree 1 and select the one with the smallest label.
- **Step 2**
 - Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.
- **Step 3**
 - Remove the chosen vertex from Step 1 and its incident edge, resulting in a smaller tree.
- **Repeat Steps 1-3**
 - Continue the process with the remaining tree until only two vertices are left.
- **Stop**
 - The process ends when only two vertices remain.
 - The resulting Prüfer sequence has now been constructed.

Example

- **Step 1**

- Identify vertices of degree 1 and select the one with the smallest label.

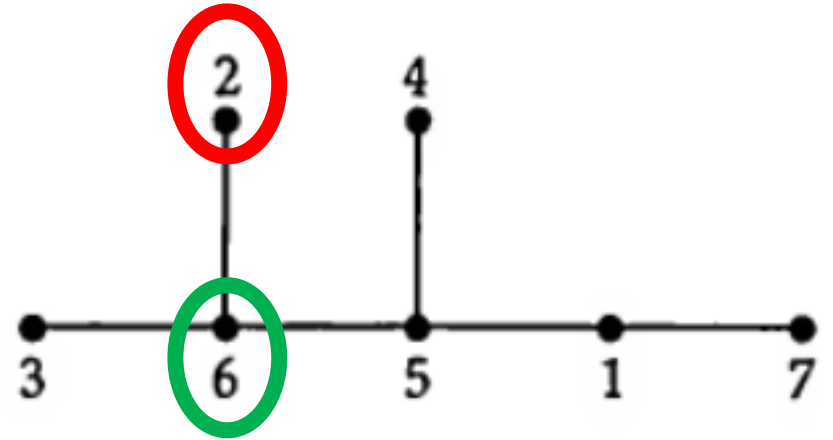
- **Step 2**

- Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.

$(6, ?, ?, ?, ?)$

- **Step 3**

- Remove the chosen vertex from **Step 1** and its incident edge, resulting in a smaller tree.



Example

- **Step 1**

- Identify vertices of degree 1 and select the one with the smallest label.

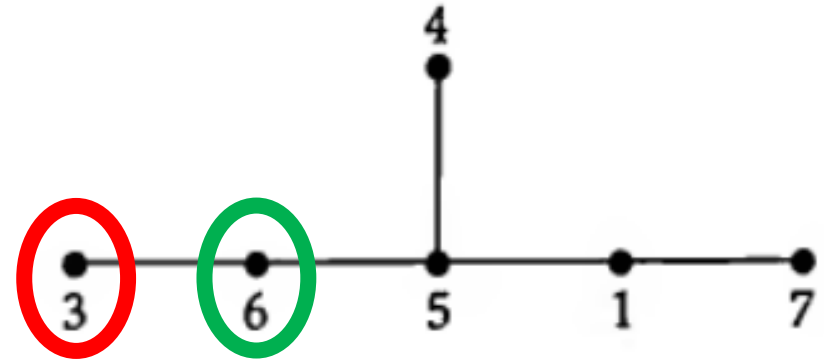
- **Step 2**

- Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.

$(6, 6, ?, ?, ?)$

- **Step 3**

- Remove the chosen vertex from **Step 1** and its incident edge, resulting in a smaller tree.



Example

- **Step 1**

- Identify vertices of degree 1 and select the one with the smallest label.

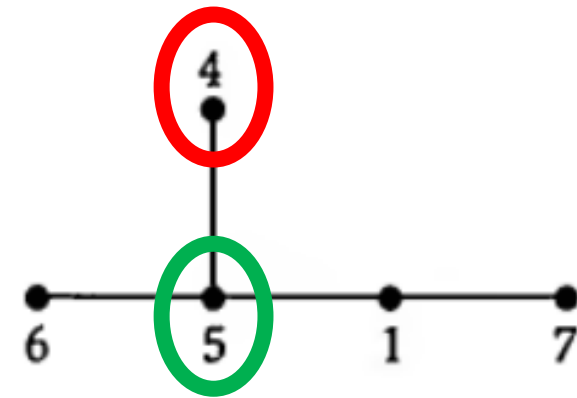
- **Step 2**

- Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.

$(6, 6, 5, ?, ?)$

- **Step 3**

- Remove the chosen vertex from **Step 1** and its incident edge, resulting in a smaller tree.



Example

- **Step 1**

- Identify vertices of degree 1 and select the one with the smallest label.

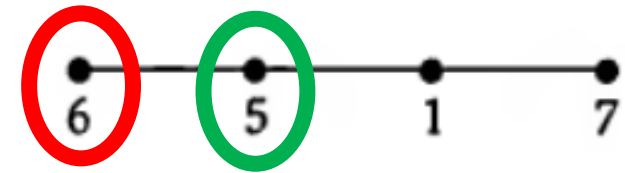
- **Step 2**

- Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.

$(6, 6, 5, 5, ?)$

- **Step 3**

- Remove the chosen vertex from **Step 1** and its incident edge, resulting in a smaller tree.



Example

- **Step 1**

- Identify vertices of degree 1 and select the one with the smallest label.

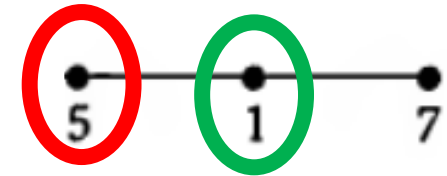
- **Step 2**

- Put the label of the vertex adjacent to the vertex of Step 1 in the next position in the Prüfer sequence.

$(6, 6, 5, 5, 1)$

- **Step 3**

- Remove the chosen vertex from **Step 1** and its incident edge, resulting in a smaller tree.



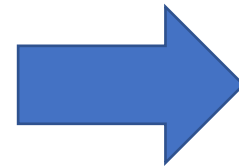
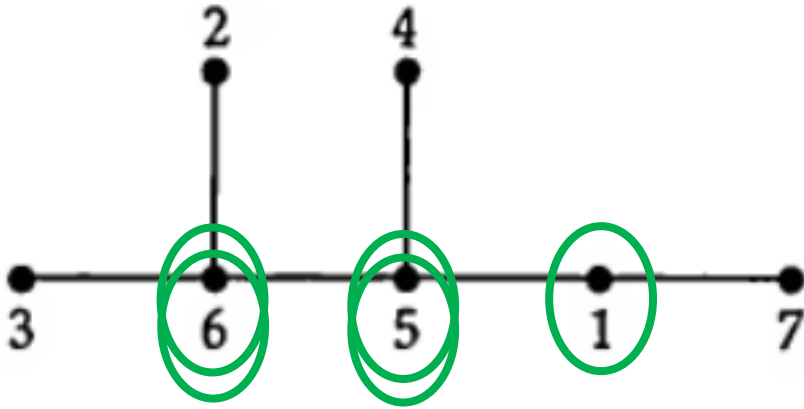
Example

- **Stop**
 - The process ends when only two vertices remain.
 - The resulting Prüfer sequence has now been constructed.

$(6, 6, 5, 5, 1)$



Example



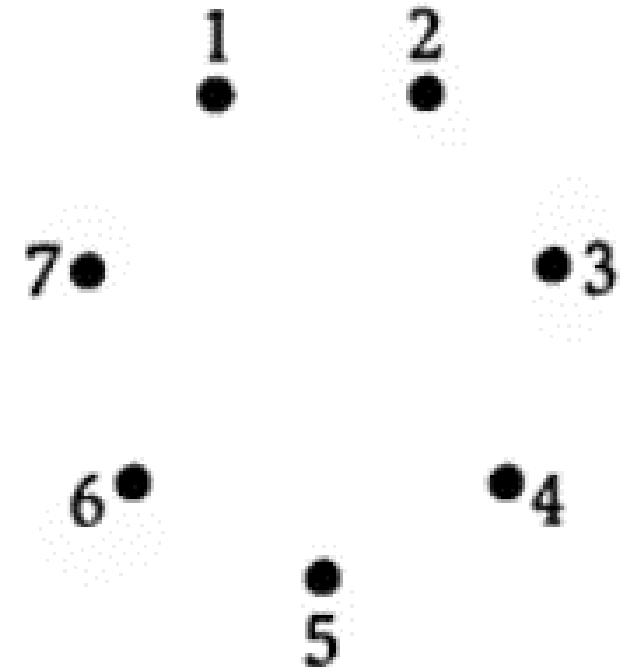
$(6,6,5,5,1)$

Constructing a Labeled Tree from a Prüfer Sequence

- **Step 1**
 - Draw ' n ' vertices, labelling them from 1 to n , and create a list of integers from 1 to n .
- **Step 2**
 - Identify the smallest list number absent in the Prüfer sequence.
 - Note the first number in the sequence.
 - Add an edge between the vertices corresponding to these numbers.
- **Step 3**
 - Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.
- **Repeat Steps 2 and 3**
 - Continue with the remaining list and sequence until only two items remain in the list.
- **Stop:**
 - Add an edge between vertices with the last two labels.
 - The corresponding labelled tree is now constructed.

Example

- **Step 1**
 - Draw 'n' vertices, labelling them from 1 to n , and create a list of integers from 1 to n .
- The list is **(1 ,2 , 3 ,4 ,5 , 6, 7).**
- The sequence is **(6, 6, 5, 5, 1).**



Example

- **Step 1**

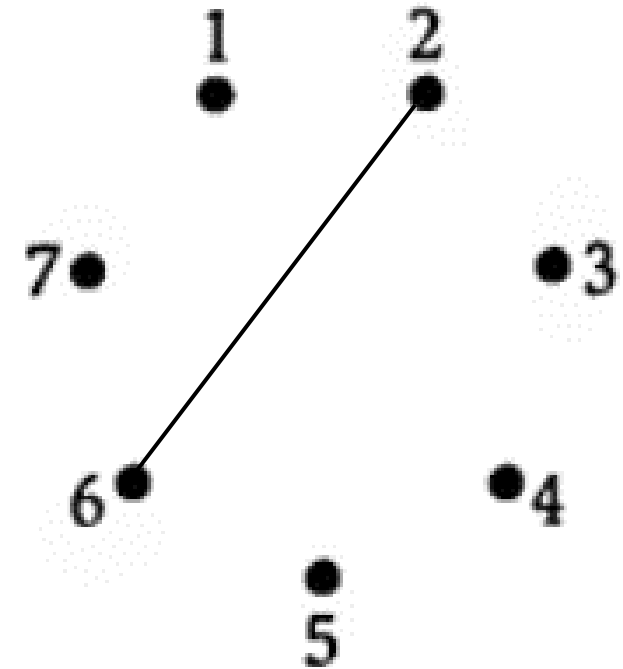
- The list is (1 , 2 , 3 ,4 ,5 , 6, 7).
- The sequence is (6, 6, 5, 5, 1).

- **Step 2**

- The smallest list number absent in the Prüfer sequence is 2.
- The first number in the sequence is 6.
- Add an edge between the vertices 2 and 6.

- **Step 3**

- Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.



Example

- **Step 1**

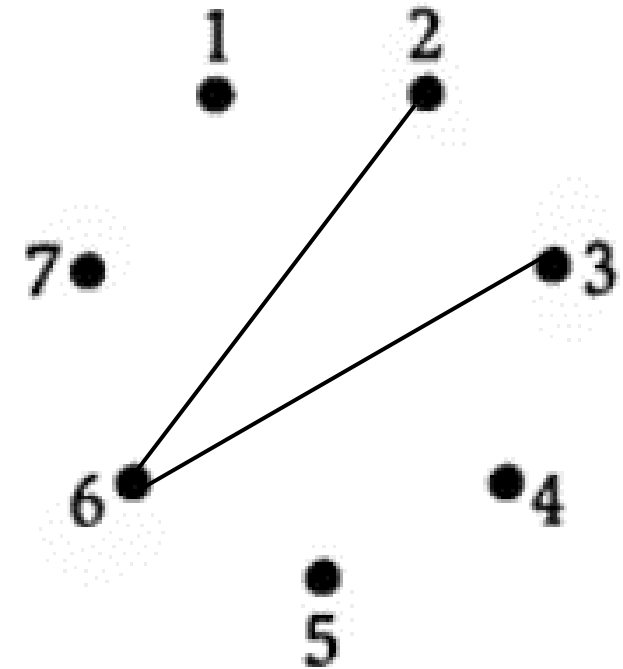
- The list is (1, 3, 4, 5, 6, 7).
- The sequence is (6, 5, 5, 1).

- **Step 2**

- The smallest list number absent in the Prüfer sequence is 3.
- The first number in the sequence is 6.
- Add an edge between the vertices 3 and 6.

- **Step 3**

- Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.



Example

- **Step 1**

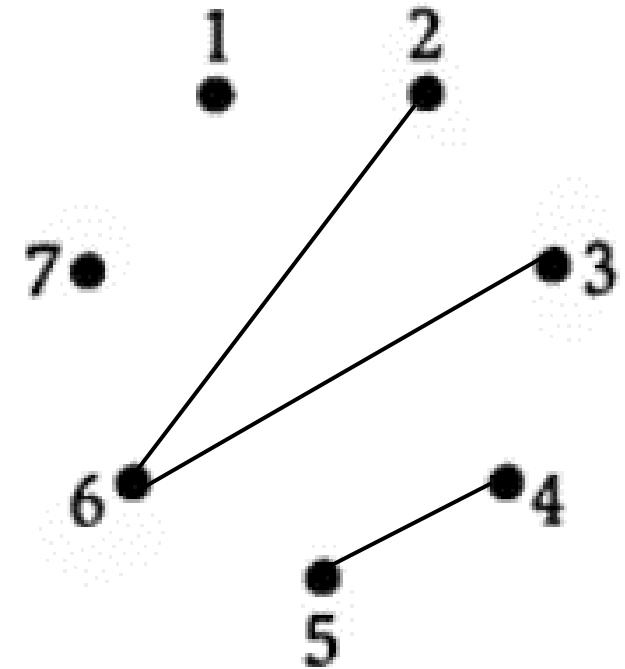
- The list is (1, 4, 5, 6, 7).
- The sequence is (5, 5, 1).

- **Step 2**

- The smallest list number absent in the Prüfer sequence is 4.
- The first number in the sequence is 5.
- Add an edge between the vertices 4 and 5.

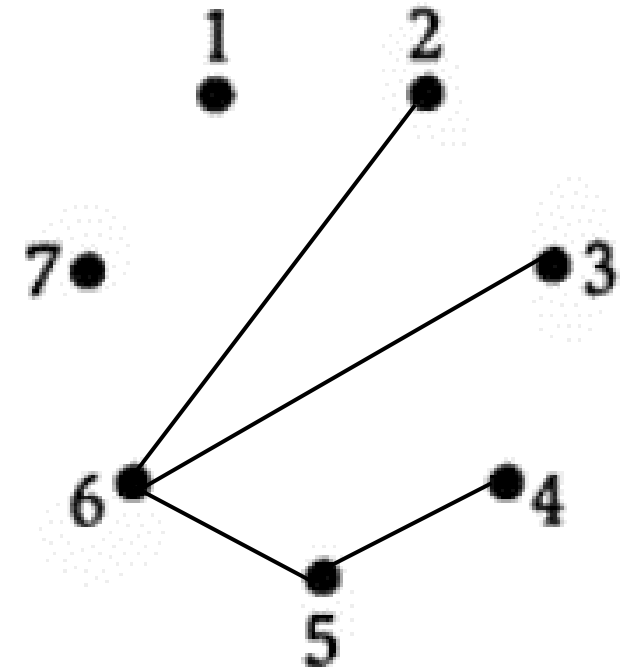
- **Step 3**

- Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.



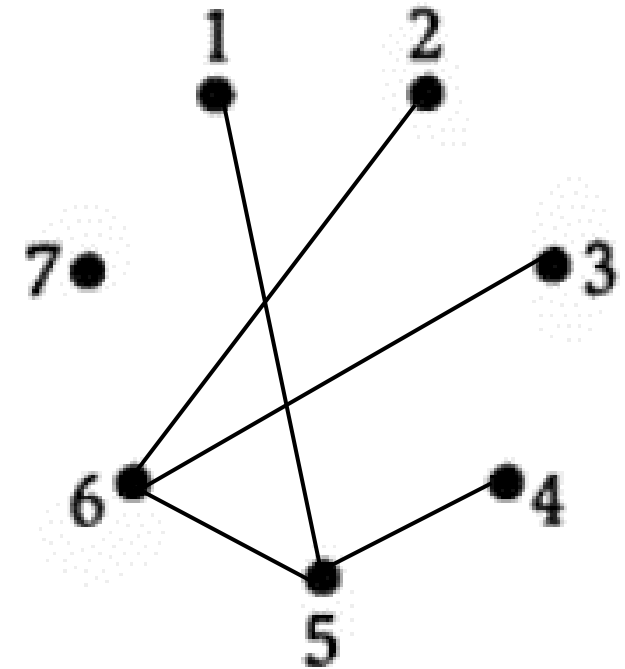
Example

- **Step 1**
 - The list is (1, 5, 6, 7).
 - The sequence is (5, 1).
- **Step 2**
 - The smallest list number absent in the Prüfer sequence is 6.
 - The first number in the sequence is 5.
 - Add an edge between the vertices 6 and 5.
- **Step 3**
 - Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.



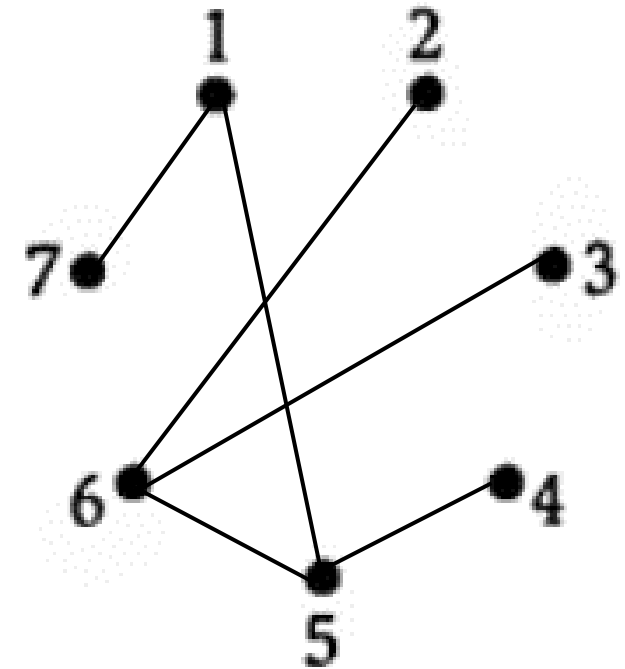
Example

- **Step 1**
 - The list is (1, 5, 7).
 - The sequence is (1).
- **Step 2**
 - The smallest list number absent in the Prüfer sequence is 5.
 - The first number in the sequence is 1.
 - Add an edge between the vertices 5 and 1.
- **Step 3**
 - Remove the first number found in Step 2 from the list and the second from the sequence, resulting in a smaller list and sequence.



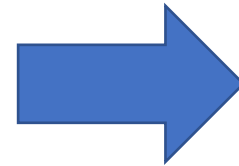
Example

- The list is **(1 , 7)**.
- The sequence is **(1)**.
- **Stop:**
 - Add an edge between vertices with the last two labels.
 - The corresponding labelled tree is now constructed.



Example

$(6, 6, 5, 5, 1)$



Enumerating Prüfer sequence

- Prüfer sequences $(a_1, a_2, a_3, \dots, a_{n-2})$,
 - a_i is one of the integers $1, 2, 3, \dots, n$ (allowing repetition)
- $n \times n \times n \times \dots \times n$ $(n - 2 \text{ times})$
- n^{n-2}

COROLLARY 1: Spanning Trees in Complete Graphs

The number of spanning trees of K_n is n^{n-2} .

Intuition:

- Every distinct tree with n vertices can form a spanning tree in a complete graph with n vertices.
- A complete graph has all possible edges between its vertices, facilitating any conceivable tree structure.

Conclusion

- Trees are fundamental structures, having the minimum required edges for connectivity.
 - Removing any single edge disrupts the tree's connectivity.
 - Adding an edge creates a cycle, transitioning the tree into a cyclic graph.
- Graph enumeration can be complex due to diverse configurations.
- Labelling vertices simplifies the problem, making counting labelled trees easier.
- Counting labelling trees is solved by Cayley's formula:

References

